KAUSORA TECHNOLOGIES

Intel® RealSense™ ROS 2 Setup Guide

Depth Camera Integration for Industrial Automation & Robotics

Document	Date	Classification
Client Setup & Deployment Guide	March 2026	Client Distribution

Kausora Technologies GitHub

https://github.com/Kausora-Technologies/kausora-realsense-ros2-guide

KAUSORA Technologies

© 2025–2026 Kausora Technologies · Masdar City, Abu Dhabi, UAE · kausora.com

Table of Contents

1. Introduction 2
2. Supported Hardware 2
3. Prerequisites 3
   3.1 System Requirements 3
   3.2 USB & Permissions 3
4. Installation 4
   4.1 Install ROS 2 Jazzy (Ubuntu 24.04) 4
   4.2 Install Intel RealSense SDK 2.0 5
   4.3 Install RealSense ROS 2 Wrapper 6
   4.4 Build the Kausora Package 6
5. Quick Start & Verification 7
6. Configuration Reference 8
   6.1 Kausora Launch Arguments 8
   6.2 Multi-Camera Setup 9
7. Docker Setup 10
   7.1 Build and Run 10
   7.2 USB Access & Verification 11
8. Common Use Cases 11
   8.1 Depth Streaming & Point Clouds 11
   8.2 SLAM & Navigation 11
   8.3 Depth Processing with Python (Example Node) 12
9. Troubleshooting 13
10. References & Resources 15

1. Introduction

This guide provides step-by-step instructions for deploying Intel® RealSense™ depth cameras with ROS 2 (Robot Operating System 2) on Ubuntu 24.04 LTS with ROS 2 Jazzy Jalisco. It is designed by Kausora Technologies for industrial automation, quality inspection, and robotics clients across the MENA region and beyond.

Intel RealSense depth cameras provide robots and automation systems with a three-dimensional view of the world. Combined with ROS 2, they enable capabilities such as real-time 3D mapping, obstacle avoidance, pick-and-place operations, quality inspection, and autonomous navigation.

What this guide covers:

- Hardware selection and compatibility matrix for the six supported camera models
- Automated and manual installation of the RealSense SDK 2.0 and ROS 2 wrapper
- Building and using the Kausora pre-configured ROS 2 package (kausora_realsense_ros2)
- Configuration for single and multi-camera deployments
- Docker-based deployment for containerized environments
- Common use cases include SLAM, point clouds, and depth processing.
- Troubleshooting for the most common deployment issues

Important: The RealSense open-source repositories have migrated from IntelRealSense to the realsenseai GitHub organization. All URLs in this guide reflect the updated locations.

2. Supported Hardware

The following Intel RealSense cameras are supported by this package. Each has a dedicated launch file, YAML config, and pre-configured RViz2 layout included in the repository. Choose the model that best fits your project's depth range, environment, and connectivity requirements.

Camera	Type	USB	Depth	Color	IMU	Fisheye
D415	Stereo Active IR	3.0	Yes	Yes	No	No
D435	Stereo Global Shutter	3.0	Yes	Yes	No	No
D435i	Stereo + IMU	3.0	Yes	Yes	Yes	No
D455	Long-range Stereo + IMU	3.0	Yes	Yes	Yes	No
L515	LiDAR Depth	3.2	Yes	Yes	No	No
T265	Visual-Inertial Tracking	2.0+	No	No	Yes	Yes

The D435i is the most commonly deployed camera for general-purpose robotics. It integrates a built-in BMI055 IMU alongside stereo depth and RGB sensors, making it well-suited for SLAM and navigation applications. Each depth pixel represents distance in millimeters as a 16-bit integer.

The T265 is a tracking-only camera (no depth output). It publishes 6-DoF pose estimates via visual-inertial odometry and is typically used alongside a depth camera to provide global localization.

3. Prerequisites

3.1 System Requirements

Component	Requirement
Operating System	Ubuntu 24.04 LTS
ROS 2 Distribution	Jazzy Jalisco: required.
Kernel	6.8+ (Ubuntu 24.04 default kernel)
USB	USB 3.0 or higher (USB 3.2 recommended for L515)
RAM	8 GB minimum · 16 GB recommended for multi-camera
Storage	20 GB free (40 GB recommended)
Python	3.12 (Ubuntu 24.04 default)

Warning: Ubuntu 22.04 and ROS 2 Humble are not supported by this package or its install scripts. Ubuntu 24.04 with ROS 2 Jazzy is required.

3.2 USB & Permissions

RealSense cameras require proper USB access permissions.

Verify USB 3.0 detection

After connecting your camera:

```
lsusb | grep -i intel
# Example: Bus 002 Device 003: ID 8086:0b3a Intel Corp. RealSense Depth Camera 435i

# Confirm SuperSpeed (USB 3.x) — look for 5000M or 10000M
lsusb -t
dmesg | grep -i 'super speed'
```

udev Rules

The librealsense SDK installer installs udev rules automatically via the librealsense2-udev-rules package. If you encounter permission errors:

```
# Reinstall udev rules
sudo apt-get install -y librealsense2-udev-rules
sudo udevadm control --reload-rules
sudo udevadm trigger
# Then reconnect the camera

# Add your user to the plugdev group
sudo usermod -aG plugdev $USER
# Log out and back in for the group change to take effect
```

4. Installation

The installation process has four steps: installing ROS 2 Jazzy, installing the Intel RealSense SDK 2.0, installing the ROS 2 camera wrapper, and building the Kausora package. Automated scripts are provided in the companion repository for steps 2 and 3.

4.1 Install ROS 2 Jazzy (Ubuntu 24.04)

Run the following commands in a terminal:

```
# Set locale
sudo apt-get update && sudo apt-get install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

# Add ROS 2 repository
sudo apt-get install -y software-properties-common
sudo add-apt-repository universe
sudo apt-get update && sudo apt-get install -y curl
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
    -o /usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) \
    signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
    http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main"
\
    | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

# Install ROS 2 Jazzy Desktop (includes RViz2, rqt)
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y ros-jazzy-desktop

# Install development tools
sudo apt-get install -y python3-colcon-common-extensions python3-rosdep
sudo rosdep init && rosdep update

# Source ROS 2 (add to ~/.bashrc for persistence)
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

4.2 Install Intel RealSense SDK 2.0

Option A — Automated Script (Recommended)

From the repository root, run the provided install script. It verifies Ubuntu 24.04, adds the Intel RealSense apt repository, and installs all required packages:

```
chmod +x scripts/install_sdk.sh
./scripts/install_sdk.sh
```

Option B — Manual Installation

```
sudo apt-get update && sudo apt-get install -y curl gnupg2 ca-certificates

# Add signing key
sudo mkdir -p /etc/apt/keyrings
 curl -sSf https://librealsense.realsenseai.com/Debian/librealsenseai.asc \
   | gpg --dearmor | sudo tee /etc/apt/keyrings/librealsenseai.gpg > /dev/null

# Add repository
    echo "deb [signed-by=/etc/apt/keyrings/librealsenseai.gpg]
    https://librealsense.realsenseai.com/Debian/apt-repo $(lsb_release
    -cs) main" \
    | sudo tee /etc/apt/sources.list.d/librealsense.list

    sudo apt-get update

# Install SDK packages
sudo apt-get install -y librealsense2           # core SDK
sudo apt-get install -y librealsense2-utils     # CLI tools (rs-enumerate-devices, etc.)
sudo apt-get install -y librealsense2-dev       # headers (for C++ development)
sudo apt-get install -y librealsense2-udev-rules  # USB permissions
```

Warning: Do not install librealsense2-dkms on Ubuntu 24.04 with a standard kernel — it is not needed and may cause issues. It is only a last resort for specific older-kernel errors (uvcvideo: Failed to query ...). See docs/02-sdk-installation.md for details.

Verify Installation

```
# List connected cameras
rs-enumerate-devices

# Check installed version
dpkg -l librealsense2

# Open the graphical viewer (requires display)
realsense-viewer
```

4.3 Install RealSense ROS 2 Wrapper

Option A — Automated Script (Recommended)

```
chmod +x scripts/install_ros2_deps.sh
./scripts/install_ros2_deps.sh
```

Option B — Manual

```
sudo apt-get update
sudo apt-get install -y \
    ros-jazzy-realsense2-camera \
    ros-jazzy-realsense2-description \
    ros-jazzy-rviz2 \
    ros-jazzy-imu-tools \
    ros-jazzy-image-view
```

4.4 Build the Kausora Package

Clone the repository into your ROS 2 workspace and build the Kausora package:

```
mkdir -p ~/kausora_ws/src && cd ~/kausora_ws/src
git clone https://github.com/Kausora-Technologies/kausora-realsense-ros2-guide.git

cd ~/kausora_ws
source /opt/ros/jazzy/setup.bash

# Install package dependencies
rosdep install --from-paths src --ignore-src -r -y

# Build
colcon build --packages-select kausora_realsense_ros2 \
    --cmake-args -DCMAKE_BUILD_TYPE=Release

# Source the workspace (add to ~/.bashrc for persistence)
echo "source ~/kausora_ws/install/setup.bash" >> ~/.bashrc
source install/setup.bash
```

5. Quick Start & Verification

With all components installed, connect your RealSense camera via USB 3.0 and launch it using the Kausora pre-configured launch files. Each camera has its own launch file with sensible. defaults:

```
# D415 — wide-angle stereo
ros2 launch kausora_realsense_ros2 d415.launch.py

# D435 — global shutter stereo
ros2 launch kausora_realsense_ros2 d435.launch.py

# D435i — stereo + IMU (SLAM/VIO)
ros2 launch kausora_realsense_ros2 d435i.launch.py

# D455 — long-range stereo + IMU
ros2 launch kausora_realsense_ros2 d455.launch.py

# L515 — LiDAR depth
ros2 launch kausora_realsense_ros2 l515.launch.py

# T265 — visual-inertial tracking
ros2 launch kausora_realsense_ros2 t265.launch.py
```

Verify Topics are Publishing

```
ros2 topic list

# Expected topics for D435 (camera_name defaults to 'd435'):
/camera/d435/color/image_raw
/camera/d435/color/camera_info
/camera/d435/depth/image_rect_raw
/camera/d435/depth/camera_info
/camera/d435/extrinsics/depth_to_color

# Additional topics for D435i (IMU enabled by default):
/camera/d435i/imu
/camera/d435i/gyro/sample
/camera/d435i/accel/sample

# T265 tracking topics:
/camera/t265/odom/sample
/camera/t265/fisheye1/image_raw
/camera/t265/fisheye2/image_raw
/camera/t265/imu
```

Launch with RViz2

Each launch file includes a launch_rviz argument that opens RViz2 with a pre-configured display layout — no manual setup required:

```
ros2 launch kausora_realsense_ros2 d435.launch.py launch_rviz:=true

# Works for all supported cameras:
ros2 launch kausora_realsense_ros2 d435i.launch.py launch_rviz:=true
ros2 launch kausora_realsense_ros2 d455.launch.py launch_rviz:=true
```

Run the Verification Script

```
python3 scripts/verify_setup.py
```

The script checks the SDK installation, connected cameras, and ROS 2 package availability, and prints a summary of any issues found.

6. Configuration Reference

6.1 Kausora Launch Arguments

All Kausora launch files accept the following common arguments. Defaults are pre-tuned per camera model (see docs/06-camera-configs.md for per-model values).

Argument	Default	Description
camera_name	model (e.g. d435)	Node and topic prefix
camera_namespace	camera	ROS namespace
serial_no	""	Camera serial number (empty = first found)
enable_pointcloud	false	Enable PointCloud2 output
align_depth.enable	false	Align depth frame to colour frame
launch_rviz	false	Open RViz2 with pre-configured displays
unite_imu_method	linear_interpolation	IMU fusion (D435i / T265 only)

Example with common options:

```
ros2 launch kausora_realsense_ros2 d435i.launch.py \
    camera_name:=kausora_cam \
    enable_pointcloud:=true \
    align_depth.enable:=true \
    launch_rviz:=true
```

Tip: Depth pixel values are 16-bit integers representing distance in millimetres. Divide by 1000.0 to convert to metres.

Per-Camera Default Resolutions

Camera	Colour (W×H@FPS)	Depth (W×H@FPS)	IMU Default
D415	1280×720@30	1280×720@30	—
D435	848×480@30	848×480@30	—
D435i	848×480@30	848×480@30	Gyro 200 Hz / Accel 63 Hz
D455	1280×720@30	1280×720@30	Gyro 400 Hz / Accel 250 Hz (off by default)
L515	1920×1080@30	1024×768@30	—
T265	848×800@30 (fisheye)	—	Always active

6.2 Multi-Camera Setup

For deployments requiring multiple cameras, use multi_camera.launch.py — a dedicated launch file that runs two cameras simultaneously, each with its own namespace and serial number. Identify your cameras first:

```
# Find serial numbers
rs-enumerate-devices | grep Serial

# Launch two cameras with unique names and serial numbers
ros2 launch kausora_realsense_ros2 multi_camera.launch.py \
    camera1_serial:=111111111111 \
    camera2_serial:=222222222222

# Optionally set custom names (topics become /camera/front/... and /camera/rear/...)
ros2 launch kausora_realsense_ros2 multi_camera.launch.py \
    camera1_name:=front  camera1_serial:=111111111111 \
    camera2_name:=rear   camera2_serial:=222222222222
```

Tip: Replace 111111111111 and 222222222222 with the 12-digit serial numbers reported by rs-enumerate-devices. Connect cameras to separate USB controllers (not just separate ports on the same hub) to avoid bandwidth contention. Run lsusb -t to verify USB topology.

rs-enumerate-devices. Connect cameras to separate USB controllers (not just separate ports on

7. Docker Setup

The repository includes a full Docker setup for running RealSense cameras in containers. This is the recommended approach for production deployments — no host SDK or ROS 2 installation is required beyond Docker itself.

7.1 Build and Run

Install Docker

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
# Log out and back in, then verify:
docker compose version
```

Configure Environment

```
cp docker/.env.example docker/.env
# Edit docker/.env:
#   CAMERA_MODEL=d435         # camera model to launch
#   CAMERA_SERIAL=            # serial number (leave blank for first found)
#   ROS_DOMAIN_ID=0           # match your network
```

Build and Run (from docker/ directory)

```
cd docker

# Build production image
docker compose build realsense

# Run (camera node publishes topics on host via network_mode: host)
docker compose up realsense

# Override camera model at runtime
CAMERA_MODEL=d455 docker compose up realsense

# Run in background
docker compose up -d realsense
```

Development Shell

```
# Build dev image (extends production image)
docker compose build dev

# Launch interactive dev shell
docker compose up dev

# Or attach to a running dev container
docker exec -it kausora_realsense_dev bash
```

7.2 USB Access & Verification

RealSense cameras require USB passthrough. The compose file handles this via privileged: true and device mapping. Verify the camera is visible inside the container:

```
docker exec kausora_realsense rs-enumerate-devices

# Verify topics are visible on the host (with ROS 2 sourced)
source /opt/ros/jazzy/setup.bash
ros2 topic list
```

Tip: privileged: true is required for USB device access inside Docker. ROS 2 topics are visible on the host because the compose file uses network_mode: host. Ensure ROS_DOMAIN_ID matches on the host and inside the container.

---

8. Common Use Cases

8.1 Depth Streaming & Point Clouds

The RealSense ROS 2 driver publishes depth, colour, and infrared streams as standard ROS topics. Point cloud generation overlays colour data onto the 3D depth representation, enabling rich spatial analysis for inspection and mapping applications.

```
# Stream depth at 1280x720@30fps with aligned colour point cloud (D455 example)
ros2 launch kausora_realsense_ros2 d455.launch.py \
    enable_pointcloud:=true \
    align_depth.enable:=true \
    launch_rviz:=true

# D435 / D435i default to 848x480 — enable point cloud:
ros2 launch kausora_realsense_ros2 d435.launch.py \
    enable_pointcloud:=true
```

8.2 SLAM & Navigation

For autonomous navigation, the depth image can be converted to a laser scan using the depthimage_to_laserscan package, which integrates with the Nav2 navigation stack and SLAM tools like Cartographer.

```
# Install depth-to-laserscan converter
sudo apt install -y ros-jazzy-depthimage-to-laserscan

# Launch depth-to-laserscan bridge (using D435i topics)
ros2 launch depthimage_to_laserscan \
    depthimage_to_laserscan-launch.py \
```

```
    depth:=/camera/d435i/depth/image_rect_raw \
    depth_camera_info:=/camera/d435i/depth/camera_info

    # Launch with mapping
     ros2 launch depthimage_to_laserscan depthimage_to_laserscan-launch.py \
     --ros-args \
     -r depth:=/camera/d435i/depth/image_rect_raw \
     -r depth_camera_info:=/camera/d435i/depth/camera_info \
     -r scan:=/scan
```

8.3 Depth Processing with Python (Example Node)

The following example shows a minimal ROS 2 node that subscribes to the depth topic, converts the image using cv_bridge, and reads the centre-pixel distance:

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge

class DepthProcessor(Node):
    def __init__(self):
        super().__init__("depth_processor")
        self.bridge = CvBrid:contentReference[oaicite:13]{index=13}create_subscription(
            Image,
            "/camera/d435/depth/image_rect_raw",
            self.callback,
            10,
        )

    def callback(self, msg: Image):
        depth = self.bridge.imgmsg_to_cv2(msg, desired_encoding="passthrough")
        h, w = depth.shape
        center_mm = int(depth[h // 2, w // 2])
        self.get_logger().info(f"Center distance: {center_mm/1000.0:.3f} m")

def main():
    rclpy.init()
    node = DepthProcessor()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

9. Troubleshooting

Camera Not Detected

Symptom: rs-enumerate-devices returns nothing or hangs; lsusb shows no Intel device.

```
lsusb | grep -i intel          # confirm camera is visible to OS
dmesg | tail -30 | grep -i usb # check for USB errors

# Reload udev rules
sudo udevadm control --reload-rules && sudo udevadm trigger
# Unplug and replug the camera

# Add user to plugdev group (not video)
sudo usermod -aG plugdev $USER
```

```
# Log out and back in
```

LIBUSB_ERROR_ACCESS / Permission Denied

Symptom: Failed to get device list. (LIBUSB_ERROR_ACCESS)

```
sudo apt-get install -y librealsense2-udev-rules
sudo udevadm control --reload-rules && sudo udevadm trigger

# Verify udev rules are installed:
ls /etc/udev/rules.d/ | grep realsense
```

ROS 2 Node Starts but No Topics Published

```
# Verify camera is detected before launching ROS 2
rs-enumerate-devices

# Inspect logs in a second terminal
ros2 run rqt_console rqt_console

# Launch with explicit serial number
ros2 launch kausora_realsense_ros2 d435.launch.py \
    serial_no:=$(rs-enumerate-devices | grep Serial | head -1 | awk '{print $NF}')
```

Node Fails: 'failed to set power state'

This indicates a USB communication issue.

```
# Unplug and replug the camera
# Use a powered USB 3.0 hub if using a laptop
# Ensure no other application is accessing the camera
ros2 launch kausora_realsense_ros2 d435.launch.py initial_reset:=true
```

USB Bandwidth Issues (Multi-Camera)

Two cameras at 1280x720@30 can saturate a single USB controller. Connect cameras to separate USB controllers, not just separate ports. Reduce resolution and frame rate if both cameras cannot stream simultaneously.

```
lsusb -t | grep -A3 Intel     # verify cameras are on different root hubs

# Reduce resolution in config/d435_config.yaml:
#   color_width: 848  color_height: 480  color_fps: 15.0
#   depth width: 848  depth height: 480  depth fps: 15.0
```

Docker: Camera Not Visible in Container

```
docker exec kausora_realsense lsusb | grep Intel

# Ensure compose file has:
#   privileged: true
#   devices:
#     - /dev/bus/usb:/dev/bus/usb
```

IMU Data Not Published (D435i / D455)

Ensure enable_gyro: true and enable_accel: true in the relevant config file, and that unite_imu_method is set for a fused /imu topic.

TF Transform Errors

If you see 'Could not find a connection between frames' in RViz2, ensure the camera node is running and the Fixed Frame in RViz2 is set to the camera_link frame (e.g. d435_link). The realsense2_description package publishes the TF tree.

```
# Check published TF frames
ros2 run tf2_tools view_frames

# Verify the camera_link frame is being published
ros2 topic echo /tf_static --once
```

Getting Help

If you cannot resolve an issue, collect the following and contact support:

```
python3 scripts/verify_setup.py   # automated check
rs-enumerate-devices
ros2 --version
dpkg -l 'librealsense2*'
uname -r
```

- Email: contact@kausora.com
- GitHub Issues: https://github.com/Kausora-Technologies/kausora-realsense-ros2-guide/issues

10. References & Resources

RealSense ROS 2 Wrapper

https://github.com/realsenseai/realsense-ros

LibRealSense SDK 2.0

https://github.com/realsenseai/librealsense

ROS 2 Documentation

https://docs.ros.org

ROS 2 Jazzy Release Notes

https://docs.ros.org/en/jazzy/Releases.html

Kausora Technologies GitHub

https://github.com/Kausora-Technologies/kausora-realsense-ros2-guide

Firmware Updates

https://dev.intelrealsense.com/docs/firmware-updates

Kausora Technologies · Robotics · AI · Automation for Industry

Masdar City, Abu Dhabi, UAE · kausora.com

For technical support, contact your Kausora project lead or email contact@kausora.com